



UNIVERSIDADE ESTÁCIO DE SÁ
PROGRAMAÇÃO FULL STACK
PÓLO NORTE SHOPPING

CLEYDSON ASSIS COELHO

MISSÃO PRÁTICA 1 MUNDO 3

RPG0014 - Iniciando o caminho pelo Java
Procedimento 2

RIO DE JANEIRO

2025

Objetivos da prática

1. Utilizar herança e polimorfismo na definição de entidades.
2. Utilizar persistência de objetos em arquivos binários.
3. Implementar uma interface cadastral em modo texto.
4. Utilizar o controle de exceções da plataforma Java.
5. No final do projeto, o aluno terá implementado um sistema cadastral em Java,
6. utilizando os recursos da programação orientada a objetos e a persistência em
7. arquivos binários.

JavaApplication.java

```
package javaapplication;
import model.*;
import java.util.Scanner;
public class JavaApplication {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        PessoaFisicaRepo repoFisica = new PessoaFisicaRepo();
        PessoaJuridicaRepo repoJuridica = new PessoaJuridicaRepo();

        int opcao;
        do {
            System.out.println("=====");
            System.out.println("1 - Incluir Pessoa");
            System.out.println("2 - Alterar Pessoa");
            System.out.println("3 - Excluir Pessoa");
            System.out.println("4 - Buscar pelo Id");
            System.out.println("5 - Exibir Todos");
            System.out.println("6 - Persistir Dados");
            System.out.println("7 - Recuperar Dados");
            System.out.println("0 - Finalizar Programa");
            System.out.println("=====");
            System.out.print("Escolha uma opção: ");
            opcao = scanner.nextInt();
            scanner.nextLine();

            switch (opcao) {
                case 1 -> {
                    System.out.println("F – Pessoa Fisica | J – Pessoa Juridica");
                    String tipo = scanner.nextLine().toUpperCase();
                    System.out.print("Digite o id da pessoa: ");
                    int id = scanner.nextInt(); scanner.nextLine();
                    System.out.println("Insira os dados...");
                    System.out.print("Nome: ");
                    String nome = scanner.nextLine();
                    if (tipo.equals("F")) {
                        System.out.print("CPF: ");
                        String cpf = scanner.nextLine();
                        System.out.print("Idade: ");
                        int idade = scanner.nextInt(); scanner.nextLine();
                        repoFisica.inserir(new PessoaFisica(id, nome, cpf, idade));
                    } else if (tipo.equals("J")) {
                        System.out.print("CNPJ: ");
                        String cnpj = scanner.nextLine();
                        repoJuridica.inserir(new PessoaJuridica(id, nome, cnpj));
                    }
                }
            }
        }
    }
}
```

```

case 2 -> {
    System.out.println("F – Pessoa Fisica | J – Pessoa Juridica");
    String tipo = scanner.nextLine().toUpperCase();
    System.out.print("Digite o id da pessoa: ");
    int id = scanner.nextInt(); scanner.nextLine();
    if (tipo.equals("F")) {
        PessoaFisica pf = repoFisica.obter(id);
        if (pf != null) {
            pf.exibir();
            System.out.println("Insira os novos dados...");
            System.out.print("Nome: ");
            String nome = scanner.nextLine();
            System.out.print("CPF: ");
            String cpf = scanner.nextLine();
            System.out.print("Idade: ");
            int idade = scanner.nextInt(); scanner.nextLine();
            repoFisica.alterar(new PessoaFisica(id, nome, cpf, idade));
        } else {
            System.out.println("Pessoa Física não encontrada.");
        }
    } else if (tipo.equals("J")) {
        PessoaJuridica pj = repoJuridica.obter(id);
        if (pj != null) {
            pj.exibir();
            System.out.println("Insira os novos dados...");
            System.out.print("Nome: ");
            String nome = scanner.nextLine();
            System.out.print("CNPJ: ");
            String cnpj = scanner.nextLine();
            repoJuridica.alterar(new PessoaJuridica(id, nome, cnpj));
        } else {
            System.out.println("Pessoa Jurídica não encontrada.");
        }
    }
}

case 3 -> {
    System.out.println("F – Pessoa Fisica | J – Pessoa Juridica");
    String tipo = scanner.nextLine().toUpperCase();
    System.out.print("Digite o id da pessoa: ");
    int id = scanner.nextInt(); scanner.nextLine();
    if (tipo.equals("F")) {
        repoFisica.excluir(id);
    } else if (tipo.equals("J")) {
        repoJuridica.excluir(id);
    }
}

case 4 -> {
    System.out.println("F – Pessoa Fisica | J – Pessoa Juridica");

```

```

String tipo = scanner.nextLine().toUpperCase();
System.out.print("Digite o id da pessoa: ");
int id = scanner.nextInt(); scanner.nextLine();
if (tipo.equals("F")) {
    PessoaFisica pf = repoFisica.obter(id);
    if (pf != null) pf.exibir();
    else System.out.println("Pessoa Física não encontrada.");
} else if (tipo.equals("J")) {
    PessoaJuridica pj = repoJuridica.obter(id);
    if (pj != null) pj.exibir();
    else System.out.println("Pessoa Jurídica não encontrada.");
}
}
case 5 -> {
    System.out.println("F – Pessoa Fisica | J – Pessoa Juridica");
    String tipo = scanner.nextLine().toUpperCase();
    if (tipo.equals("F")) {
        for (PessoaFisica pf : repoFisica.obterTodos()) pf.exibir();
    } else if (tipo.equals("J")) {
        for (PessoaJuridica pj : repoJuridica.obterTodos()) pj.exibir();
    }
}
case 6 -> {
    System.out.print("Digite o prefixo dos arquivos: ");
    String prefixo = scanner.nextLine();
    try {
        repoFisica.persistir(prefixo + ".fisica.bin");
        repoJuridica.persistir(prefixo + ".juridica.bin");
        System.out.println("Dados salvos com sucesso.");
    } catch (Exception e) {
        System.out.println("Erro ao salvar dados: " + e.getMessage());
    }
}
case 7 -> {
    System.out.print("Digite o prefixo dos arquivos: ");
    String prefixo = scanner.nextLine();
    try {
        repoFisica.recuperar(prefixo + ".fisica.bin");
        repoJuridica.recuperar(prefixo + ".juridica.bin");
        System.out.println("Dados recuperados com sucesso.");
    } catch (Exception e) {
        System.out.println("Erro ao recuperar dados: " + e.getMessage());
    }
}
case 0 -> System.out.println("Encerrando o sistema...");
default -> System.out.println("Opção inválida.");
}
} while (opcao != 0);

```

```
}  
}
```

Pessoa.java

```
package model;  
import java.io.Serializable;  
public class Pessoa implements Serializable {  
    private int id;  
    private String nome;  
    public Pessoa() {  
    }  
    public Pessoa(int id, String nome) {  
        this.id = id;  
        this.nome = nome;  
    }  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public String getNome() {  
        return nome;  
    }  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
    public void exibir() {  
        System.out.println("Id: " + id);  
        System.out.println("Nome: " + nome);  
    }  
}
```

PessoaFisica.java

```
package model;  
import java.io.Serializable;  
public class PessoaFisica extends Pessoa implements Serializable {  
    private String cpf;  
    private int idade;  
    public PessoaFisica() {  
        super();  
    }  
    public PessoaFisica(int id, String nome, String cpf, int idade) {  
        super(id, nome);  
    }  
}
```

```

        this.cpf = cpf;
        this.idade = idade;
    }
    public String getCpf() {
        return cpf;
    }
    public void setCpf(String cpf) {
        this.cpf = cpf;
    }
    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }
    @Override
    public void exibir() {
        System.out.println("Id: " + getId());
        System.out.println("Nome: " + getNome());
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }
}

```

PessoaFisicaRepo.java

```

package model;
import java.io.*;
import java.util.ArrayList;
public class PessoaFisicaRepo {
    private ArrayList<PessoaFisica> lista = new ArrayList<>();

    public void inserir(PessoaFisica pessoa) {
        lista.add(pessoa);
    }
    public void alterar(PessoaFisica pessoa) {
        for (int i = 0; i < lista.size(); i++) {
            if (lista.get(i).getId() == pessoa.getId()) {
                lista.set(i, pessoa);
                return;
            }
        }
    }
    public void excluir(int id) {
        lista.removeIf(p -> p.getId() == id);
    }
}

```

```

public PessoaFisica obter(int id) {
    for (PessoaFisica p : lista) {
        if (p.getId() == id) {
            return p;
        }
    }
    return null;
}
public ArrayList<PessoaFisica> obterTodos() {
    return lista;
}
public void persistir(String nomeArquivo) throws Exception {
    FileOutputStream fos = new FileOutputStream(nomeArquivo);
    ObjectOutputStream oos = new ObjectOutputStream(fos);
    oos.writeObject(lista);
    oos.close();
    fos.close();
}
public void recuperar(String nomeArquivo) throws Exception {
    FileInputStream fis = new FileInputStream(nomeArquivo);
    ObjectInputStream ois = new ObjectInputStream(fis);
    lista = (ArrayList<PessoaFisica>) ois.readObject();
    ois.close();
    fis.close();
}
}

```

PessoaJuridicaRepo.java

```

package model;
import java.io.*;
import java.util.ArrayList;
public class PessoaJuridicaRepo {
    private ArrayList<PessoaJuridica> lista = new ArrayList<>();

    public void inserir(PessoaJuridica pessoa) {
        lista.add(pessoa);
    }
    public void alterar(PessoaJuridica pessoa) {
        for (int i = 0; i < lista.size(); i++) {
            if (lista.get(i).getId() == pessoa.getId()) {
                lista.set(i, pessoa);
                return;
            }
        }
    }
    public void excluir(int id) {
        lista.removeIf(p -> p.getId() == id);
    }
}

```



```

    }
    public PessoaJuridica obter(int id) {
        for (PessoaJuridica p : lista) {
            if (p.getId() == id) {
                return p;
            }
        }
        return null;
    }
    public ArrayList<PessoaJuridica> obterTodos() {
        return lista;
    }
    public void persistir(String nomeArquivo) throws Exception {
        FileOutputStream fos = new FileOutputStream(nomeArquivo);
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(lista);
        oos.close();
        fos.close();
    }
    public void recuperar(String nomeArquivo) throws Exception {
        FileInputStream fis = new FileInputStream(nomeArquivo);
        ObjectInputStream ois = new ObjectInputStream(fis);
        lista = (ArrayList<PessoaJuridica>) ois.readObject();
        ois.close();
        fis.close();
    }
}

```

PessoaJuridica.java

```
package model;
```

```
import java.io.Serializable;
```

```

public class PessoaJuridica extends Pessoa implements Serializable {
    private String cnpj;

    public PessoaJuridica() {
        super();
    }

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    public String getCnpj() {
        return cnpj;
    }
}

```

```

    }

    public void setCnpj(String cnpj) {
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        System.out.println("Id: " + getId());
        System.out.println("Nome: " + getNome());
        System.out.println("CNPJ: " + cnpj);
    }
}

```

Resultados:

```

run:
=====
1 - Incluir Pessoa
2 - Alterar Pessoa
3 - Excluir Pessoa
4 - Buscar pelo Id
5 - Exibir Todos
6 - Persistir Dados
7 - Recuperar Dados
0 - Finalizar Programa
=====
Escolha uma opção:

```

Análise e Conclusão:

1. O que são elementos estáticos e qual o motivo para o método main adotar esse modificador?

Resposta:

Elementos estáticos em Java são aqueles que pertencem à classe e não aos objetos. Isso significa que eles podem ser usados sem precisar criar uma instância da classe.

O método main é estático porque é o ponto de entrada do programa, e o Java precisa chamá-lo diretamente, antes mesmo de qualquer objeto ser criado.

2. Para que serve a classe Scanner?

Resposta:

A classe Scanner serve para ler dados digitados pelo usuário, como textos e números. Ela é usada para interagir com o teclado e capturar entradas no console.

No programa, usamos o Scanner para ler opções do menu e os dados que o usuário digita (como nome, CPF, CNPJ, idade, etc.).

3. Como o uso de classes de repositório impactou na organização do código?

Resposta:

As classes de repositório (PessoaFisicaRepo e PessoaJuridicaRepo) ajudaram a organizar melhor o código, deixando claro que a responsabilidade de armazenar, alterar, excluir e recuperar os dados das pessoas está separada da lógica do menu.

Isso torna o sistema mais modular, fácil de manter, testar e reaproveitar no futuro.